

DD2352 Algorithms and Complexity
Spring 2026
Lecture 8: Randomized Algorithms

Per Austrin

2026-03-16

Agenda

Randomized Algorithms

Min-Cut

Max-Cut

Randomized Algorithms

A randomized algorithm is an algorithm that uses randomness to make some of its choices.

Such an algorithm will typically have some (hopefully small) probability of failing.

We distinguish two ways in which such failures can happen:

- ▶ Could be a risk that the algorithm outputs the *wrong answer*.
- ▶ Could be a risk that the algorithm runs for much *longer time* than expected.

Randomized Algorithms vs Average Case

It is important to note that we are still talking about algorithms which work on *all possible instances*

The only thing that is random is the algorithm's "internal" choices, not the input.

In other words we are still interested in the *worst case behaviour of the algorithm on any possible instance*, not in the average case behaviour on a typical instance.

Las Vegas Algorithms

A **Las Vegas** algorithm is a randomized algorithm that always finds the correct answer, but the running time of the algorithm might vary significantly depending on the random choices made.

For Las Vegas algorithms, we look at the **expected** running time of the algorithm over the random choices made.

(The expectation is only over the “internal” random choices that the algorithm makes, the input to the algorithm is still worst-case.)

Monte Carlo Algorithms

A **Monte Carlo** algorithm is a randomized algorithm where the output of the algorithm may be incorrect.

For these algorithms we want some form of guarantee about the error. For example an upper bound on the *error probability* of algorithm, or a bound on the *expected value* of the output.

(These bounds are only over the “internal” random choices that the algorithm makes, the input to the algorithm is still worst-case.)

Illustration: Guessing Pin Code

Suppose you have forgotten the 4-digit pin code to your KTH access card and want to figure it out.

1. A *Las Vegas* algorithm for this could be to randomly guess pin codes (without remembering what you already tried) until you guess the right one.

If you are unlucky it is possible that you could keep going indefinitely, but in expectation you will find the right pin code after a mere 10 000 attempts.

2. A *Monte Carlo* algorithm for this could be to do the same thing, but after you have made 5 000 attempts you give up.

The probability that you fail to find the code is $\left(\frac{9999}{10000}\right)^{5000} \approx 0.6$.

(Randomness is not very helpful here; neither of these algorithms is better than simply testing the 10 000 possible codes in order.)

Aside: Where Does Randomness Come From?

We think of randomized algorithms as algorithms that make choices at random by flipping a coin.

Clearly there is no actual physical coin to flip inside our computers.

In a computer, we use a **Random Number Generator** (RNG).

- ▶ True RNG: gets randomness from measuring physical phenomena (e.g. background radiation) that we *believe* is sufficiently random.
- ▶ Pseudo-RNG: starting from a small random *seed*, new numbers are generated in a completely deterministic way.

When we talk about randomized algorithms we analyze them as if we had access to perfect randomness.

It is generally theoretically possible that when we run our algorithm on the output of an RNG it behaves completely differently.

(But if it does, it means the RNG is broken in some way and this may be much more interesting than whatever algorithm we were working on!)

Agenda

Randomized Algorithms

Min-Cut

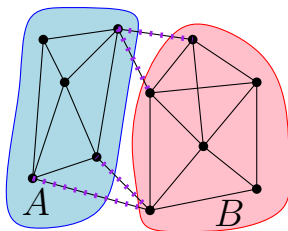
Max-Cut

The Minimum Cut Problem

(GLOBAL) MIN-CUT

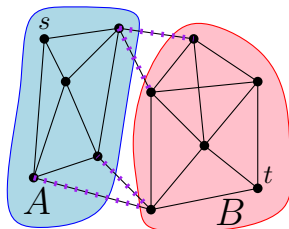
Input: Graph G

Output: Partition of vertices of G into two non-empty sets A and B such that number of edges between A and B is minimized.



Global Min-Cut vs. s - t -Min-Cut

MIN-CUT is very similar to the s - t -Cut problem that we saw earlier in the course. There we were additionally given two vertices s and t and the cut (A, B) needed to separate s from t .



Global Min-Cut vs. s - t -Min-Cut

MIN-CUT is very similar to the s - t -Cut problem that we saw earlier in the course. There we were additionally given two vertices s and t and the cut (A, B) needed to separate s from t .

We also saw that an optimal s - t -cut can be found using max-flow.

This immediately gives a way of solving MIN-CUT:

1. Fix some vertex u of the graph
2. For every vertex $v \neq u$, find a minimum u - v -Cut
3. Return the best of all the cuts found

We call the s - t -Cut solver $n - 1$ times, so if solving s - t -Cut takes T time then this algorithm for MIN-CUT takes $O(nT)$ time.

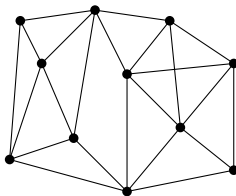
Can we solve MIN-CUT faster than solving s - t -Cut n times??

Constructing Cuts by Contraction

We are going to repeatedly **contract** edges in the graph.

This means we take an edge and merge its two vertices.

Example:

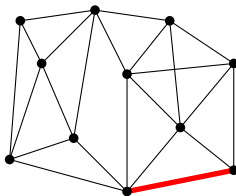


Constructing Cuts by Contraction

We are going to repeatedly **contract** edges in the graph.

This means we take an edge and merge its two vertices.

Example:

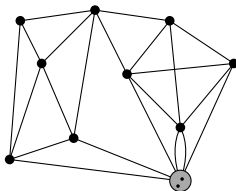


Constructing Cuts by Contraction

We are going to repeatedly **contract** edges in the graph.

This means we take an edge and merge its two vertices.

Example:

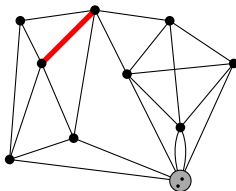


Constructing Cuts by Contraction

We are going to repeatedly **contract** edges in the graph.

This means we take an edge and merge its two vertices.

Example:

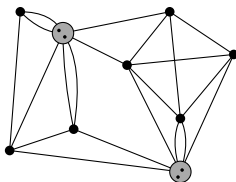


Constructing Cuts by Contraction

We are going to repeatedly **contract** edges in the graph.

This means we take an edge and merge its two vertices.

Example:



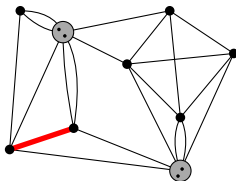
The vertices in the contracted graph correspond to sets of original vertices that we have grouped together.

Constructing Cuts by Contraction

We are going to repeatedly **contract** edges in the graph.

This means we take an edge and merge its two vertices.

Example:

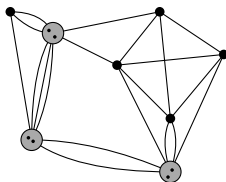


Constructing Cuts by Contraction

We are going to repeatedly **contract** edges in the graph.

This means we take an edge and merge its two vertices.

Example:



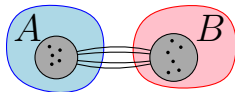
Note that the contracted graph will typically have many edges between the same pair of vertices!

Constructing Cuts by Contraction

We are going to repeatedly **contract** edges in the graph.

This means we take an edge and merge its two vertices.

Example:



We stop when there are only two vertices left in the graph.
These two groups of vertices is our partition of the vertices.

Randomized Min-Cut Algorithm

How to choose which edges to contract?

Let us try the following very simple randomized strategy:

```
1 function RandomizedMinCut( $G$ ):  
2   while  $G$  has more than 2 vertices  
3     | Pick a uniformly random edge of  $G$  and contract it.  
4   return the resulting cut
```

This algorithm seems so simple that it cannot possibly work, but it is surprisingly good.

We need to analyze the success probability of the algorithm:
how likely is it to find a correct answer (a minimum cut)?

Analyzing one step of the Algorithm

Fix any minimum cut. What is the probability that the very first choice we make is compatible with this cut?

Let $m = \# \text{edges in graph}$ and $k = \text{value of minimum cut}$.

First choice of algorithm is bad only if we pick one of the k edges from the optimal cut. This happens with probability k/m .

Each vertex in the graph must have degree at least k (why?), and therefore we must have $m \geq \frac{kn}{2}$, where n is the number of vertices.

So the first random edge we contract is bad with probability $k/m \leq 2/n$ which is pretty low.

In i 'th step of the algorithm, we have $n - i + 1$ vertices left and then the algorithm chooses a good edge with probability at least

$$1 - \frac{2}{n - i + 1} = \frac{n - i - 1}{n - i + 1}$$

Analyzing the Full Algorithm

Choices made in each iteration are independent, so the probability that we choose a good edge in all $n - 2$ iterations is

$$\frac{n-2}{n} \cdot \frac{n-3}{n-1} \cdot \frac{n-4}{n-2} \cdot \frac{n-5}{n-3} \cdots \frac{3}{5} \cdot \frac{2}{4} \cdot \frac{1}{3} = \frac{2}{n(n-1)} \geq \frac{2}{n^2}$$

I.e., algorithm finds a minimum cut with probability at least $2/n^2$.

Sounds small, but can improve by running algorithm many times.

If we repeat r times with independent random choices, the probability that all r runs fail to find a minimum cut is at most

$$(1 - 2/n^2)^r \leq e^{-r \cdot 2/n^2} \quad (\text{because } 1 - x \leq e^{-x} \text{ for all } x)$$

If we set $r = n^2/2$ then failure probability is at most $1/e \approx 0.37$.

(And if we want smaller failure probability we only have to increase r a bit e.g. with $r = 2n^2$ the failure probability is only $1/e^4 \approx 0.02$.)

Runtime Analysis

Each run of the contraction algorithm can be implemented in $O(m)$ time (not immediately obvious, just take my word for it)

With $r = n^2/2$ the *total runtime is $O(n^2m)$* .

This algorithm is **simpler** than the one based on computing many s - t cuts. But **not faster** (if using a fast max-flow algorithm).

However the algorithm can be refined and running time improved to $O(n^2 \log(n))$ (relatively simple algorithm) or $O(m \log^3(n))$ (more complicated algorithm)
(See Exercise 6 for some parts of this.)

Much faster than just running an s - t -Min-Cut algorithm n times!

(Recently, improvements to this 25-year-old algorithm were found by Danupon Nanongkai and Sagnik Mukhopadhyay from KTH!)

Agenda

Randomized Algorithms

Min-Cut

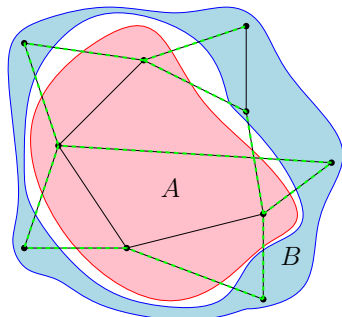
Max-Cut

The Maximum Cut Problem

MAX-CUT

Input: Graph G

Output: Partition of vertices of G into two non-empty sets A and B such that number of edges between A and B is maximized.



11 of the 15 edges are cut

Trying to solve Max-Cut

We can try various approaches to solving Max-Cut

- ▶ Greedy
- ▶ Divide & Conquer
- ▶ Dynamic Programming
- ▶ Using Max-Flows
- ▶ ...or more advanced techniques not covered in this course

But they all seem to fail.

It turns out there is a reason for this:

The Max-Cut problem is a so called **NP-hard** problem.

We will talk a lot about **NP-hardness** in the coming weeks, but takeaway message for now: *unlikely that there exists an efficient algorithm to solve Max-Cut.*

Trying to solve Max-Cut

Let us lower the ambition and try to find a fast algorithm that finds a “reasonably good” cut.

Consider the following completely trivial algorithm:

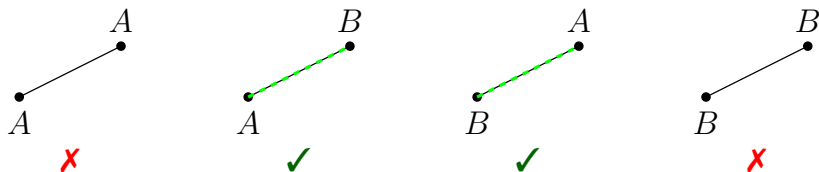
Put each vertex in either A or B independently with probability $1/2$.

Could this silly algorithm actually do something meaningful?

Let us analyze!

Analyzing the Random Assignment

For each edge, there are 4 possible assignments to the two vertices:



Exactly 2 of these 4 possibilities results in the edge being cut.

Each of these 4 outcomes are equally likely.

So the random assignment cuts the edge with probability $1/2$.

Analyzing the Random Assignment (Cont.)

Define:

- ▶ $m = \text{\#edges}$
- ▶ Z_i be a random variable which equals 1 if i 'th edge gets cut by the algorithm, and 0 otherwise.
- ▶ $\mathbf{Alg} = \sum_i Z_i$ be the total number of edges cut by algorithm.

Using linearity of expectation, expected value of $\mathbf{Alg}$ is

$$\mathbb{E}[\mathbf{Alg}] = \mathbb{E}\left[\sum_{i=1}^m Z_i\right] = \sum_{i=1}^m \mathbb{E}[Z_i] = \sum_{i=1}^m \Pr[i\text{'th edge is cut}] = \frac{1}{2}m$$

In other words, the random assignment algorithm cuts (in expectation) at least half of all the edges in the graph.

Max-Cut Summary

The “silly” random assignment algorithm (which does not even look at the graph!) cuts in expectation at least half of the edges in the graph.

The optimum clearly cannot cut more than all the edges, so optimum cannot be *too* much better (at most 2x higher).

This is an example of an **approximation algorithm**, which is the topic of a later lecture.

Upcoming Week In The Course

- ▶ Tomorrow 15-17: Lecture 9
Complexity Intro and Models of Computation
- ▶ Thursday 8-10: Exercise 6
Randomized Algorithms